University of Papua New Guinea
School of Natural and Physical Sciences
Department of Mathematics, Statistics and Computer Science
Address: PO Box 320 University PO Waigani, Port Moresby, Phone: 326 7414, Email: secmaths@upng.ac.pg

Python Dictionary

University of Papua New Guinea
School of Natural and Physical Sciences
Department of Mathematics, Statistics and Computer Science
Address: PO Box 320 University PO Waigani, Port Moresby, Phone: 326 7414, Email: secmaths@upng.ac.pg

- A Python dictionary is a collection of key-value pairs where each key is associated with a value.
- A value in the key-value pair can be a number, a string, a list, a tuple, or even another dictionary.
- In fact, you can use a value of any valid type in Python as the value in the key-value pair.
- A key in the key-value pair must be immutable.
- In other words, the key cannot be changed, for example, a number, a string, a tuple, etc.
- Python uses the curly braces {} to define a dictionary.
- Inside the curly braces, you can place zero, one, or many key-value pairs.

Example defines an empty dictionary:

```
empty_dict = {}

print(type(empty_dict))
```

Ouptut:

```
<class 'dict'>
```

```
person = {
    'first_name': 'John',
    'last_name': 'Doe',
    'age': 25,
    'favorite_colors': ['blue', 'green'],
    'active': True
}
```

University of Papua New Guinea
School of Natural and Physical Sciences
Department of Mathematics, Statistics and Computer Science
Address: PO Box 320 University PO Waigani, Port Moresby, Phone: 326 7414, Email: secmaths@upng.ac.pg

Accessing values in a Dictionary:

1) Using square bracket notation:

```
person = {
        'first_name': 'John',
        'last_name': 'Doe',
        'age': 25,
        'favorite_colors': ['blue', 'green'],
        'active': True
        }

print(person['first_name'])
print(person['last_name'])
```

2) Using the get() method

```
person = {
   'first_name': 'John',
   'last_name': 'Doe',
   'age': 25,
   'favorite_colors': ['blue', 'green'],
   'active': True
}

name= person.get(‘first_name')
print(name)

#Adding new & modifying existing key-value pairs:

person['gender'] = 'Famale'
person['age'] = 26

print(person)
```

Removing key-value pairs:

```
person = {
    'first_name': 'John',
    'last_name': 'Doe',
    'age': 25,
    'favorite_colors': ['blue', 'green'],
    'active': True
}

del person['active']
print(person)
```

Looping through a dictionary:

1) Looping all key-value pairs in a dictionary:

- Python dictionary provides a method called **items**() that returns an object which contains a list of key-value pairs as tuples in a list.

For example:

```
person = {
   'first_name': 'John',
   'last_name': 'Doe',
   'age': 25,
   'favorite_colors': ['blue', 'green'],
   'active': True
}

print(person.items())
```

- **To iterate over all key-value pairs in a dictionary, you use a for loop with two variable key and value**

```
person = {
    'first_name': 'John',
    'last_name': 'Doe',
    'age': 25,
    'favorite_colors': ['blue', 'green'],
    'active': True
}
for key, value in person.items():
    print(f"{key}: {value}")
```

Output:

```
first_name: John
last_name: Doe
age: 25
favorite_colors: ['blue', 'green']
active: True
```

2) Looping through all the keys in a dictionary

```
person = {
    'first_name': 'John',
    'last_name': 'Doe',
    'age': 25,
    'favorite_colors': ['blue', 'green'],
    'active': True
}
for key in person.keys():
    print(key)
```

Output:

```
first_name
last_name
age
favorite_colors
active
```

3) Looping through all the values in a dictionary:

```
person = {
    'first_name': 'John',
    'last_name': 'Doe',
    'age': 25,
    'favorite_colors': ['blue', 'green'],
    'active': True
}

for value in person.values():
    print(value)
```

Output:

```
John
Doe
25
['blue', 'green']
True
```

University of Papua New Guinea
School of Natural and Physical Sciences
Department of Mathematics, Statistics and Computer Science
Address: PO Box 320 University PO Waigani, Port Moresby, Phone: 326 7414, Email: secmaths@upng.ac.pg

Python Exception:

Exceptions:

- Even though when your code has valid syntax, it may cause an error during execution.
- In Python, errors that occur during the execution are called exceptions.
- The causes of exceptions mainly come from the environment where the code executes. For example:
- Reading a file that doesn't exist.
- Connecting to a remote server that is offline.
- Bad user inputs.
- When an exception occurs, the program doesn't handle it automatically. This results in an error message.

University of Papua New Guinea
School of Natural and Physical Sciences
Department of Mathematics, Statistics and Computer Science
Address: PO Box 320 University PO Waigani, Port Moresby, Phone: 326 7414, Email: secmaths@upng.ac.pg

Handling exceptions:

- To make the program more robust, you need to handle the exception once it occurs.
- In other words, you need to catch the exception and inform users so that they can fix it.
- A good way to handle this is not to show what the Python interpreter returns. Instead, you replace that error message with a more user-friendly one.
- To do that, you can use the Python try...except statement:

```
try:
        # code that may cause error
except:
        # handle errors
```

The try...except statement works as follows:

- The statements in the try clause executes first.

- If no exception occurs, the except clause is skipped and the execution of the try statement is completed.

- If an exception occurs at any statement in the try clause, the rest of the clause is skipped and the except clause is executed.

- So to handle exceptions using the try...except statement, you place the code that may cause an exception in the try clause and the code that handles exception in the except clause.

University of Papua New Guinea
School of Natural and Physical Sciences
Department of Mathematics, Statistics and Computer Science
Address: PO Box 320 University PO Waigani, Port Moresby, Phone: 326 7414, Email: secmaths@upng.ac.pg

Example:

```
a = int(input("Enter a:"))
b = int(input("Enter b:"))
c = a/b
print("a/b = %d" %c)

#other code:
print(“It is an example of exception in Python!")
```

Another Example:

```
try:
    a = int(input("Enter a:"))
    b = int(input("Enter b:"))
    c = a/b
except:
    print("Can't divide with zero")
```

We can also use the else statement with the try-except statement in which, we can place the code which will be executed in the scenario if no exception occurs in the try block.

```
try:
   #block of code

except Exception1:
    #block of code

else:
    #this code executes if no except block is executed
```

```
try:
   a = int(input("Enter a:"))
   b = int(input("Enter b:"))
   c = a/b
   print("a/b = %d"%c)
   # Using exception object with the except statement
except Exception as e:
   print("can't divide by zero")
   print(e)
else:
   print("Hi I am else block")
```

University of Papua New Guinea
School of Natural and Physical Sciences
Department of Mathematics, Statistics and Computer Science
Address: PO Box 320 University PO Waigani, Port Moresby, Phone: 326 7414, Email: secmaths@upng.ac.pg

The try...finally block

- Python provides the optional finally statement, which is used with the try statement.
- It is executed no matter what exception occurs and used to release the external resource.
- The finally block provides a guarantee of the execution.

```
try:
    # block of code
    # this may throw an exception
finally:
    # block of code
    # this will always be executed
```

```
 try:
    fileptr = open("file2.txt","r")
    try:
        fileptr.write("Hi I am good")
    finally:
        fileptr.close()
        print("file closed")
except:
    print("Error")
```

University of Papua New Guinea
School of Natural and Physical Sciences
Department of Mathematics, Statistics and Computer Science
Address: PO Box 320 University PO Waigani, Port Moresby, Phone: 326 7414, Email: secmaths@upng.ac.pg

Raising exceptions:

- An exception can be raised forcefully by using the raise clause in Python.
- It is useful in in that scenario where we need to raise an exception to stop the execution of the program.

Syntax

```
raise Exception_class,<value>
```

```
try:
    num = int(input("Enter a positive integer: "))
    if(num <= 0):
# we can pass the message in the raise statement
        raise ValueError("That is  a negative number!")
except ValueError as e:
    print(e)
```